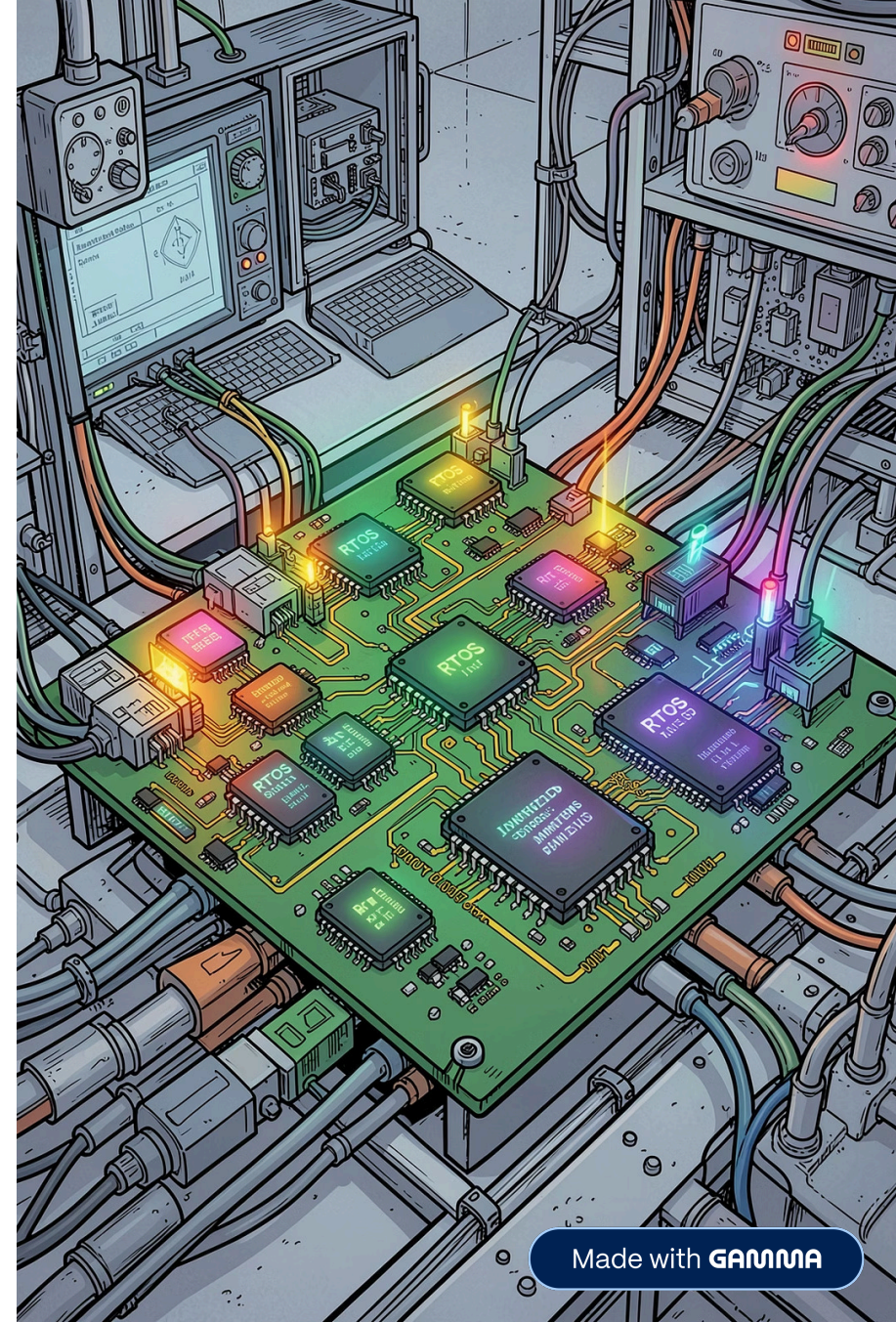


FreeRTOS Advanced Features: Modul 10

Modul 10 membahas fitur lanjutan FreeRTOS yang digunakan dalam sistem embedded industri modern. Target akhir adalah penyelesaian **25 eksperimen final** — 10 eksperimen STM32, 10 eksperimen ESP32, dan 5 eksperimen Multi STM32-ESP32 — yang dirancang untuk membangun kompetensi praktis mahasiswa dalam mengimplementasikan fitur RTOS lanjutan. Semua eksperimen diarahkan untuk mendukung pengembangan project akhir berupa **Advanced RTOS Industrial System** yang mengintegrasikan monitoring dan kontrol industri.



Struktur Eksperimen Modul 10

Setiap eksperimen diimplementasikan pada platform STM32 dan ESP32 untuk membandingkan perilaku FreeRTOS di kedua arsitektur. Eksperimen Multi mengintegrasikan kedua MCU dengan koordinasi menggunakan objek RTOS lanjutan via UART.

10

Eksperimen STM32

Implementasi fitur RTOS lanjutan pada mikrokontroler STM32F103C8/STM32F4

10

Eksperimen ESP32

Implementasi fitur RTOS lanjutan pada ESP32 dengan ESP-IDF/Arduino

5

Eksperimen Multi

Koordinasi STM32-ESP32 menggunakan objek RTOS lanjutan via UART

12

Fitur RTOS

Event Groups, Software Timers, Task Notifications, Semaphore, Mutex, Memory Management, Static Allocation, Critical Section, Message/Stream Buffers, Queue Sets

Event Groups & Software Timers

Event Groups

Objek sinkronisasi berbasis **bit flags** dengan 24 bit yang dapat di-set, cleared, atau ditunggu. Mendukung logika **AND/OR** untuk kombinasi event dan lebih ringan dari multiple semaphore.

- `xEventGroupCreate()` — membuat Event Group
- `xEventGroupSetBits()` — mengatur bit (tersedia versi ISR)
- `xEventGroupWaitBits()` — menunggu bit tertentu
- `xEventGroupClearBits()` — membersihkan bit manual

Praktikum: STM32_01 & ESP32_01 — sinkronisasi LED indikator dengan button press menggunakan bit 0 dan bit 1.

Software Timers

Timer yang dikelola kernel FreeRTOS, berjalan di **task daemon timer** dengan prioritas konfigurasi via `configTIMER_TASK_PRIORITY`. Non-blocking dan tidak memakan CPU saat menunggu.

- `xTimerCreate()` — membuat timer (one-shot atau periodik)
- `xTimerStart()` / `xTimerStop()` — kontrol timer
- `xTimerChangePeriod()` — ubah periode dinamis

Praktikum: STM32_02 & ESP32_02 — toggle LED setiap 1 detik tanpa memblokir task lain.

Task Notifications & Semaphore Varieties

Task Notifications

Mekanisme komunikasi paling ringan di FreeRTOS — menggunakan struktur pada **TCB task tujuan** tanpa objek eksplisit tambahan.

- **45% lebih cepat** dan **45% lebih hemat memory** dibanding semaphore
- Mendukung pengiriman nilai 32-bit antar task
- `xTaskNotifyGive()` / `ulTaskNotifyTake()`
- `xTaskNotify()` / `xTaskNotifyWait()` dengan nilai 32-bit

Keterbatasan: Hanya unicast — tidak mendukung broadcast seperti Event Groups.

Praktikum: STM32_03 & ESP32_03 — menggantikan binary semaphore dengan respon lebih cepat.

Semaphore Varieties

FreeRTOS menyediakan tiga jenis semaphore untuk kasus penggunaan berbeda:

Binary Semaphore

Nilai 0 atau 1. Sinkronisasi ISR ke task. Tanpa Priority Inheritance.

Counting Semaphore

Nilai 0 hingga max count. Kontrol akses multiple resource identik.

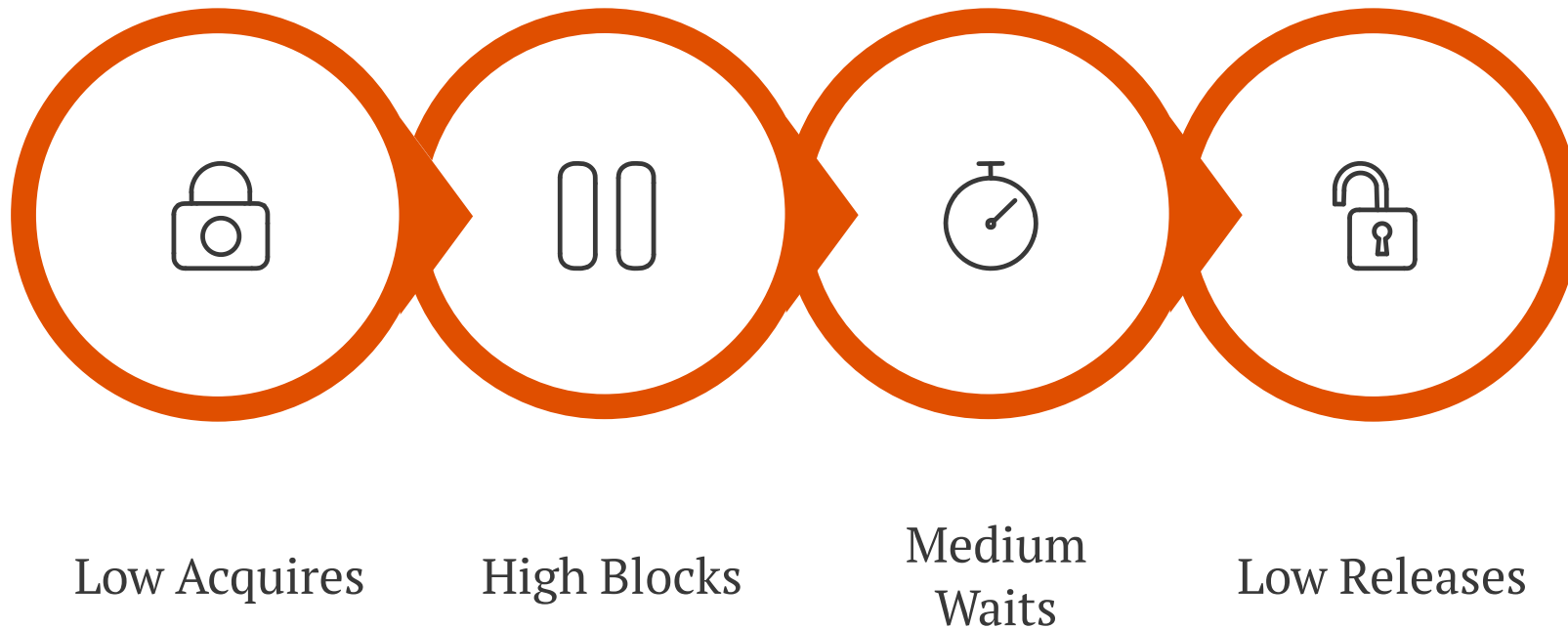
Mutex

Mirip Binary Semaphore dengan Priority Inheritance. Hanya owner yang boleh release.

Praktikum: STM32_04 & ESP32_04 — perbandingan perilaku sinkronisasi ketiga jenis.

Mutex & Priority Inheritance

Priority Inversion terjadi ketika task low priority memegang mutex, task high priority menunggu, dan task medium priority menunda task low priority — sehingga task high priority ikut tertunda. Mutex FreeRTOS menggunakan **Priority Inheritance** untuk mencegah masalah ini.



Mekanisme ini memastikan task medium tidak dapat menunda task low saat memegang mutex yang dibutuhkan task high. Pada praktikum, diuji pada STM32_05 dan ESP32_05 dengan skenario tiga task berprioritas berbeda yang memperebutkan satu mutex.

Memory Management & Static Allocation

Heap Management (5 Jenis)

1

heap_1

Alokasi saja, tidak bisa free. Untuk sistem alokasi sekali.

2

heap_2

Bisa free, fragmentasi tinggi. Tidak direkomendasi.

3

heap_3

Wrapper malloc/free standar, thread-safe.

4

heap_4

Dinamis dengan coalescence, fragmentasi rendah.

Direkomendasi umum.

5

heap_5

Multiple memory region. Untuk STM32 dengan RAM terpisah.

Fungsi terkait: `pvPortMalloc()`, `vPortFree()`, `xPortGetFreeHeapSize()`, `xPortGetMinimumEverFreeHeapSize()`. Praktikum: STM32_O6 & ESP32_O6.

Static Allocation

Membuat objek FreeRTOS dengan memory yang dialokasikan sebelumnya — **tanpa menyentuh heap dinamis**. Cocok untuk sistem deterministik dan safety-critical.

- Deklarasi `StaticTask_t` `xTaskBuffer` dan `StackType_t` `xStack[STACK_SIZE]`
- Buat task dengan `xTaskCreateStatic()`
- Queue, semaphore, timer juga memiliki versi `CreateStatic()`

Keunggulan: Eksekusi deterministik, tanpa fragmentasi heap, cocok untuk sistem safety-critical.

Praktikum: STM32_O7 & ESP32_O7 — task dan queue tanpa heap dinamis.

Critical Section & Task Suspension

Critical Section

Mekanisme proteksi data paling ringan — **menonaktifkan scheduler/interrupt sementara** agar operasi berjalan atomis. Tidak boleh memanggil RTOS API di dalamnya.

- `taskENTER_CRITICAL()` / `taskEXIT_CRITICAL()`
- Versi ISR: `taskENTER_CRITICAL_FROM_ISR()`
- Cocok untuk: increment counter, set flag — operasi sangat singkat

vs Mutex: Mutex dapat memanggil RTOS API dan mendukung Priority Inheritance; Critical Section tidak.

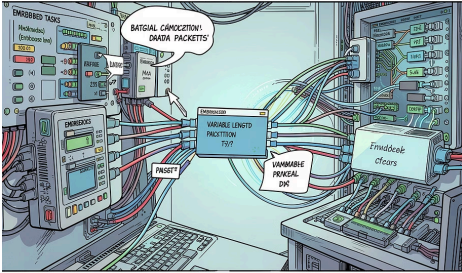
Task Suspension

Memungkinkan kontrol eksekusi task dari task lain **tanpa menghapus task**. Task yang di-suspend tetap ada di memori dan dapat diresume kapan saja.

- `vTaskSuspend()` — suspend task tertentu atau `NULL` untuk task sendiri
- `vTaskResume()` — resume task dari task lain
- `xTaskResumeFromISR()` — resume dari konteks ISR

Praktikum: STM32_O8 & ESP32_O8 — demonstrasi race condition dan perlindungan data atomis.

Message Buffers & Stream Buffers



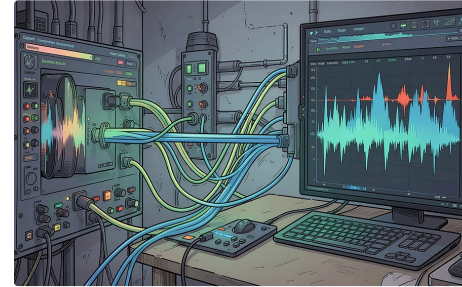
Message Buffer

Transfer data **panjang variabel** antar task atau ISR. Header panjang pesan ditambahkan otomatis saat pengiriman.

- `xMessageBufferCreate()` — membuat buffer
- `xMessageBufferSend()` — mengirim pesan (tersedia versi ISR)
- `xMessageBufferReceive()` — menerima pesan

Cocok untuk: komando, log, data JSON, data sensor dengan panjang berubah (1–128 byte).

Praktikum: STM32_09 & ESP32_09 — perbandingan perilaku Message Buffer (data variabel) dan Stream Buffer (stream kontinu).



Stream Buffer

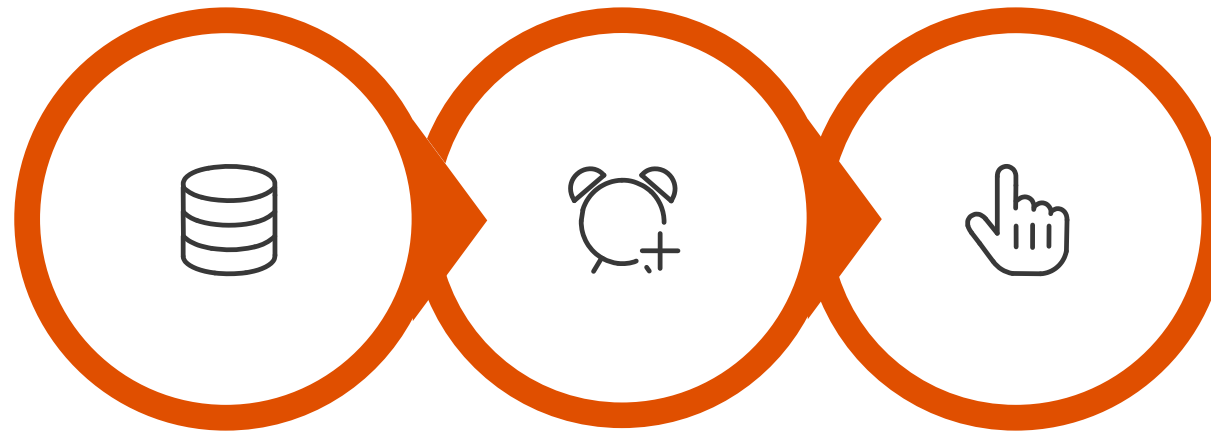
Transfer **byte stream kontinu** tanpa header panjang — hanya byte mentah. Trigger level menentukan minimum byte sebelum task penerima bangun.

- `xStreamBufferCreate(size, triggerLevel)`
- `xStreamBufferSend()` / `xStreamBufferReceive()`

Cocok untuk: audio, sensor stream, data serial kontinu.

Queue Sets

Queue Set memungkinkan satu task **menunggu event dari multiple queue, semaphore, atau mutex** dalam satu panggilan — tanpa perlu polling masing-masing objek secara manual. Menghemat CPU dan menyederhanakan arsitektur task gateway.



Create Set

Add Objects

Wait & Read

Keunggulan Queue Sets

- Menghindari polling multiple objek — menghemat CPU
- Satu task dapat menangani banyak sumber event sekaligus
- Cocok untuk task gateway yang menerima input dari queue data, semaphore event, dan notifikasi

Praktikum

STM32_10 & ESP32_10 — memantau **2 queue dan 1 semaphore** dalam satu task menggunakan Queue Set. Mahasiswa membandingkan efisiensi dengan pendekatan polling konvensional.

Platform, Pemilihan Objek RTOS & Error Handling

STM32 vs ESP32

STM32: FreeRTOS via STM32CubeHAL/STM32CubeIDE. RAM terbatas — heap_4 direkomendasi, Static Allocation untuk task kritis.

ESP32: FreeRTOS via ESP-IDF/Arduino. Dual-core (atur core via xCoreID). RAM lebih besar, namun Static Allocation tetap disarankan untuk task kritis.

Panduan Pemilihan Objek RTOS

- **Event Groups** — sinkronisasi multiple event/flags
- **Task Notifications** — komunikasi unicast paling ringan
- **Mutex** — shared resource dengan priority inheritance
- **Message/Stream Buffer** — data variabel / byte stream
- **Queue Sets** — pantau multiple objek dalam satu task

⚠ **Error Umum FreeRTOS:** Heap habis → gunakan heap_4 atau Static Allocation. Priority Inversion → gunakan Mutex, bukan Binary Semaphore. Task deadlock → hindari multiple mutex bersamaan. Buffer overrun → sesuaikan ukuran buffer dan implementasikan handling.